

Academic Year 2025 – 2026

INP - UGA

Groupe C : Aurane Bourgault, Alexandre Coumes, Le Thanh

Hien Nguyen, Elliott Poisson, Shivraj Sharode

N. Vannier, Ronald Phlypo & M. Dohen



DATA ANALYSIS

Data Challenge Report

Bourgault, Coumes, Nguyen, Poisson, Sarode

June 5, 2026

Abstract

Keywords: Image Processing, Data Analyst, Signals and Systems

Contents

1	Introduction	3
1.1	Goal	3
1.2	Our strategies	3
2	Basis	4
2.1	Data	4
2.2	Load and Visualise the Data	4
2.3	Metrics	5
3	Detection	5
3.1	Importance of detection	5
3.2	Preliminary Explorations	5
3.3	Detailed Analysis of Evaluated Techniques	6
3.4	Hyperparameter Optimization	12
3.5	Final Detection Architecture in 2 steps	13
4	Correction	13
4.1	Smoothing	14
4.2	Copy paste method	15
4.3	Inpaint Opencv	15
4.4	Non-Uniform Correction method for dealing with Fixed Pattern Noise	16
4.5	Interpolation	16
4.6	Wavelet Fourier	17
4.7	Frequency Interpolation	18
4.8	Trials of Deep Learning Models	19
5	Results	20
6	Limitations Future Perspective	20
7	Appendix : Detection Performance Metrics	21

1 Introduction

1.1 Goal

The objective of this project addresses the image quality issues found in certain infrared sensors. Specifically, these sensors exhibit three types of pixel defects: noisy, blinking, and a combination of both (noisy-blinking). Because replacing the faulty hardware amplifiers is expensive, a software-based post-processing approach was chosen as a more cost-effective solution. Our mandate was to develop an algorithm capable of providing the best possible image correction while remaining as lightweight and computationally efficient as possible.



Figure 1 – Defective image and the aimed corrected one

1.2 Our strategies

To tackle this problem, our approach was to design a modular architecture based on well-defined “boxes”, with strictly specified inputs and outputs. We deliberately chose to separate the process into two distinct phases: defect detection and image correction. By standardizing the data format between these steps : imposing a universal output format for the coordinates of defective pixels, we ensured complete interoperability. As a result, any detection algorithm we develop can be combined with any correction algorithm.

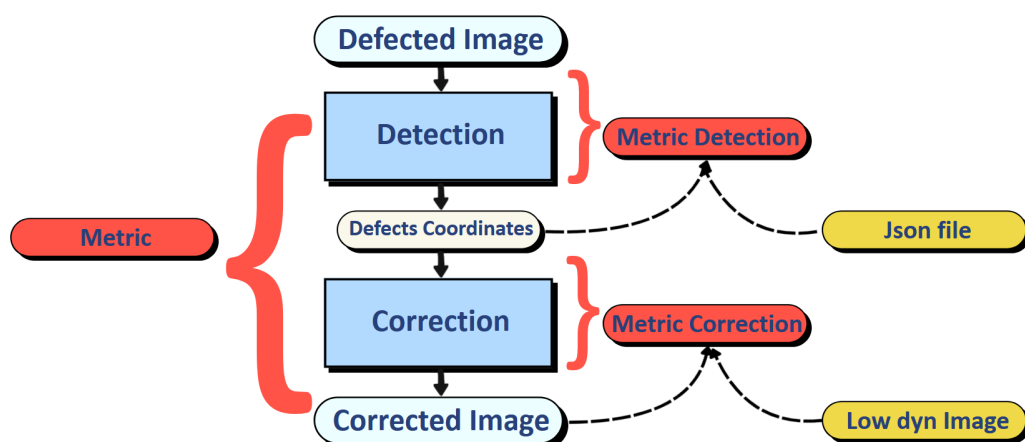


Figure 2 – Planning architecture for the development of image defect correction software

2 Basis

2.1 Data

To train and evaluate our algorithm, we were provided a dataset containing sequences from three different infrared sensors: HD, SXGA, and VGA. Each sequence consists of a video that has been extracted frame-by-frame into individual images. The data is organized into specific folders based on dynamic range and the presence of defects. The *low dyn* (low dynamics) and *high dyn* (high dynamics) folders contain clean, pristine images, which served as our baseline (ground truth). To simulate the hardware issues, the clean *low dyn* sequences were artificially corrupted to create three new sets of defective folders (labeled as 'columns 1, 2, and 3'). These correspond to the three specific hardware defect profiles: noisy, blinking, and noisy-blinking pixels.

A component of this dataset is the inclusion of a *.json* file within each defective sequence folder. Because the defects were synthetically generated and injected into the clean images, this file acts as an absolute ground truth. It precisely documents the spatial and temporal coordinates of every defective pixel for each frame. This structure allowed us to train our detection algorithms using the JSON files, and evaluate our correction algorithms by comparing our outputs directly against the original, defect-free low dyn images.

2.2 Load and Visualise the Data

Core Data Management

To manage the data extraction and evaluation processes efficiently, a set of core utility functions was implemented. First, `load_image` reads the raw sensor images from the file system and converts them into `numpy` arrays for algorithmic processing. For ground-truth management, `load_json` handles the opening and parsing of the provided JSON files associated with each defective sequence. Finally, `get_defect_coordinates` targets a specific frame within that parsed JSON data to extract the exact spatial mapping of the defects. This structures the coordinates into a standardized dictionary, allowing the evaluation pipeline to pinpoint exactly where the simulated defects are located for any given image.

The 16-bit vs. 8-bit Dilemma

Raw infrared sensors capture data in a 16-bit format, providing a massive dynamic range of intensity values. To preserve maximum precision and prevent any mathematical information loss, all core detection and correction algorithms operate strictly on the raw 16-bit arrays. The conversion to an 8-bit range (0 to 255) using the `normalize_to_8bit` function is strictly reserved for human visualisation.

Evaluated Visualization Techniques

To effectively map the 16-bit data to standard 8-bit displays, several tone-mapping strategies were tested:

- **Neural Adaptive Gain Control (N-AGC):** An advanced method that is extremely fast and preserves details well. However, it artificially pushes local contrasts to assist machine vision, which introduces unnatural visual artifacts for human observation.
- **CLAHE (Contrast Limited Adaptive Histogram Equalization):** A standard local contrast enhancement technique. While functional, it proved visually suboptimal for highlighting the specific hardware defect profiles present in this dataset.
- **Quantile-Based Normalization:** A mathematically sound approach for matrix standardization, but computationally overly complex for simple visualization purposes.

Final Visualization Strategy

Ultimately, a straightforward percentile-based clipping method was adopted for the `normalize_to_8bit` function. Using `numpy`'s `percentile()` and `clip()` methods, the algorithm identifies the relevant

intensity bounds (effectively ignoring extreme noise outliers) and scales the values strictly within this defined interval. This approach provided a clean, computationally inexpensive 8-bit image for visual analysis, without requiring complex histogram equalizations or neural networks.

2.3 Metrics

The evaluation of our algorithms was a multi-step process, beginning with the precise extraction of defective pixel data. To achieve this, we developed the `load_json` and `get_defect_coordinates` functions. The original JSON files are structured by defect type and spatial location, containing a list of frames where each defect appears, along with an associated intensity coefficient. Our extraction functions parse this data to pinpoint the exact defect coordinates for one picture. These extracted coordinates served a dual purpose in our pipeline. First, for the detection phase, they provided the exact ground truth needed to compute standard classification metrics. We evaluated our detection algorithms by calculating True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN), ultimately deriving the overall detection accuracy. Second, for the correction phase, these coordinates dictated exactly where our correction algorithms should operate. To evaluate the quality of the image restoration, we compared the corrected pixels directly against the defect-free 'low dyn' ground truth. Since evaluating image correction is not a simple binary (True/False) matter, we implemented a continuous performance scale based on grayscale intensity differences. On this scale, 0% represents the original defective pixel and 100% represents a perfect match with the ground truth. By measuring the grayscale values along the corrected lines, we calculated intermediate percentages for every processed pixel. These values were then averaged to produce a final, highly nuanced score reflecting the true quality of the correction.

3 Detection

3.1 Importance of detection

The detection phase is one of the most critical components of this project. The subsequent correction process relies entirely on the accurate identification of defective pixels. Even an optimal correction algorithm will perform poorly if it modifies healthy pixels (destroying the $RMSE_{ok}$ score) or misses defective ones (ruining the $RMSE_{def}$ score). To ensure compatibility between the detection and correction stages, a unified data structure was established. The detection output is strictly formatted to match the input requirements for both the correction tools and the evaluation metrics. This is achieved using a standardized JSON dictionary that stores the exact coordinates (x, y_{start}, y_{stop}) of every defective segment.

3.2 Preliminary Explorations

Before settling on our final architecture, several traditional image processing techniques were evaluated to better understand these defective infrared sensors.

- Morphological Transformations: Initial tests considered using morphological operations to pre-process the images and artificially enhance defects prior to detection. However, this approach risked amplifying naturally occurring vertical structures within the scene, thereby increasing the rate of false positives.
- 1D Spatial Projections: By compressing the image into a 1D graph (column index vs. intensity), we attempted to identify defects via global thresholding. We quickly realized a global threshold was impossible due to natural intensity variations across the image (see Figure 3).

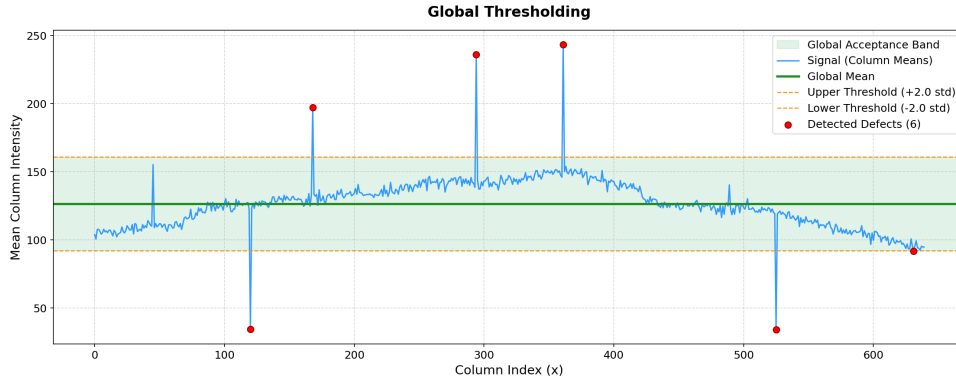


Figure 3 – Graph of Global Thresholding

3.3 Detailed Analysis of Evaluated Techniques

3.3.1 Local Thresholding (LT)

To overcome the impossibility of applying a single global threshold across an image with natural thermal gradients, a 1D Local Thresholding (LT) technique was implemented. Instead of evaluating a column against the entire image, this method evaluates each column relative to its immediate neighborhood (see Figure 4).

The algorithm slides a horizontal window of size W across the image. For each column under evaluation, it computes a local statistical baseline. During initial tests, several statistical metrics were compared. Pure variance-based evaluations were quickly discarded, as variance proved to be excessively sensitive to the inherent non-stationary noise of the infrared images, causing precision scores to drop below 3% on some sequences. In contrast, using the local mean or median provided a much more stable and reliable baseline (see Table 1 of Appendix).

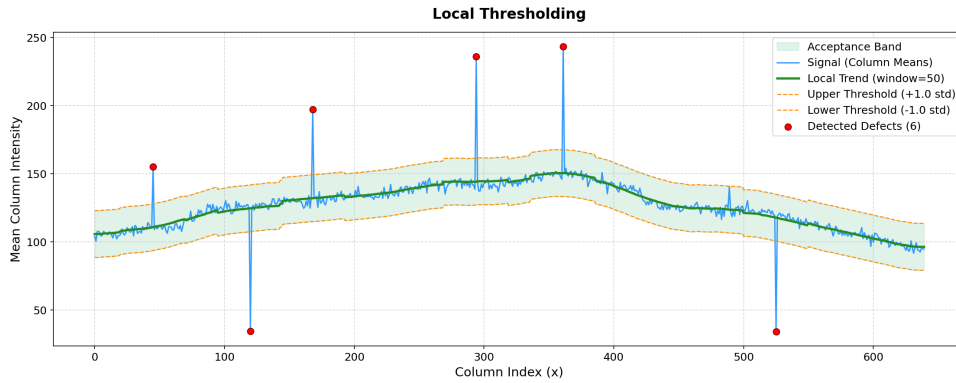


Figure 4 – Plot of a Local Threshold

A column is flagged as defective if its intensity deviates from the local baseline by more than a dynamic threshold:

$$|I_i - \mu_{local}| > k \cdot \sigma$$

where I_i is the intensity of the column, μ_{local} is the local mean, and k is an adjustable standard deviation factor.

In practice, the algorithm operates as follows

1. The column-wise mean of the image is computed to obtain a 1D signal.

2. A 1D convolution using a uniform smoothing filter of size W is applied to this signal (using a reflecting mode at the borders) to extract the local trend (μ_{local}).
3. A tolerance margin is defined using the standard deviation of the signal multiplied by the factor k .
4. Any column whose absolute difference from the local trend exceeds this margin is recorded as defective.

Strengths and Limitations

Strengths : The main advantage of the Local Thresholding approach is its robustness to structural illumination changes combined with a very low computational cost. By adapting locally, it ignores natural scene gradients that typically cause false positives. The method demonstrated high precision, reaching up to 100% on specific VGA sequences, making it a highly reliable tool for isolating the x-coordinates of suspect columns (see Table 2 in Appendix).

Limitations : Despite this high precision, the LT method presented a major structural limitation regarding the final evaluation metrics. Operating purely on a 1D column-wide basis assumes that the defect spans the entire height of the image. When applied to fragmented defects, some were too short to be detected but the method successfully flags most of the defective columns. However, correcting entire columns overwrites thousands of healthy pixels, which severely degrades the $RMSE_{ok}$ score. Consequently, while LT acts as an excellent detector for the x-coordinate, it fails to measure the exact vertical boundaries (y_start and y_stop coordinates).

To try to answer this, a “banded version” of the method was tested. The image was divided into multiple horizontal bands, and the LT algorithm was applied independently to each segment. Although this method partially succeeded in isolating defects vertically, it provided negligible performance gains over the baseline LT while significantly increasing computational costs (see Table 2 in Appendix).

3.3.2 Constant False Alarm Rate (CFAR)

To have a more adaptive approach and better handle non-stationary noise, a 1D adaptation of the Constant False Alarm Rate (CFAR) algorithm was explored. Originally designed for radar systems to detect targets against varying background clutter, CFAR calculates an adaptive threshold based on the immediate local noise.

Unlike standard sliding windows, CFAR divides the neighborhood into three distinct zones :

- **Cell Under Test (CUT):** The specific column currently being evaluated.
- **Guard Cells:** Adjacent columns immediately surrounding the CUT. These are intentionally ignored in the calculation to prevent a thick defect from artificially inflating the local average (signal spreading).
- **Training Cells:** Columns located further away, outside the guard zone. These are used to calculate the true “background noise.”

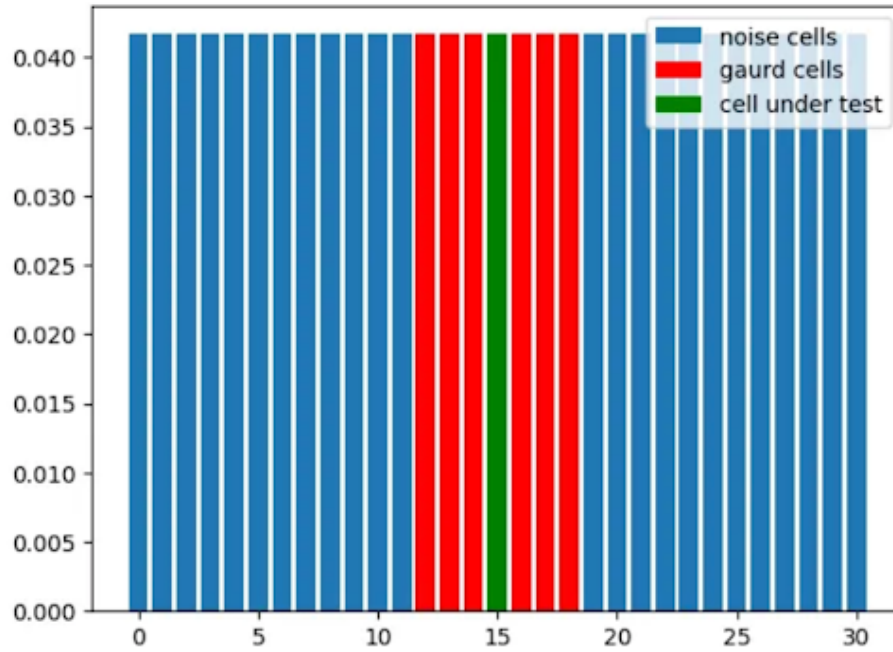


Figure 5 – CFAR cells

The detection threshold is determined by multiplying the average or median of the training cells by a scaling factor, α . In radar literature, α is calculated using the desired probability of false alarm (P_{fa}) and the number of training cells (N_{train}):

$$\alpha = N_{train} \cdot (P_{fa}^{-1/N_{train}} - 1)$$

However, testing revealed that this formula, based on radar wave physics, was unsuitable for infrared thermal images. Therefore, α was replaced simply by a tolerance multiplier. Furthermore, to handle image borders where training cells fall outside the matrix, mirroring (reflecting mode) was used, as simple zero-padding was found to artificially skew the local averages.

In practice, the algorithm operates as follows

1. The column-wise median (or mean) of the image is computed to obtain a 1D signal.
2. A custom 1D convolution kernel is constructed with equal weights for the outer Training Cells, while the central Cell Under Test (CUT) and the adjacent Guard Cells are strictly masked with zeros (For example, with two training and two guard cells on each side, the kernel is $[1, 1, 0, 0, 0, 1, 1] / 4$).
3. A 1D convolution using this custom kernel is applied to the signal (using a reflecting mode at the borders) to estimate the true local background noise for each column without signal spreading.
4. A dynamic threshold is defined by multiplying this local background noise by the scaling factor α .
5. Any column whose intensity strictly exceeds this dynamic threshold is recorded as defective.

Strengths and Limitations

Strengths : The inclusion of guard cells successfully prevented defect bleed-over from corrupting the local baseline. As a result, the standard CFAR method achieved high detection scores for defective columns that spanned the full height of the sensor.

Limitations : Similar to the standard LT method, basic CFAR evaluates the entire column, rendering it blind to the specific start and end points of fragmented defects.

To address this, a CFAR_band approach was introduced. By dividing the image into horizontal bands, the algorithm attempted to detect shorter anomalies. However, this created an unsolvable trade-off (see Figure 6): parameters tuned to be sensitive enough to detect small fragmented defects caused full-length defects to be incorrectly split into smaller pieces. Conversely, configurations optimized to detect full columns completely missed the fragmented defects.

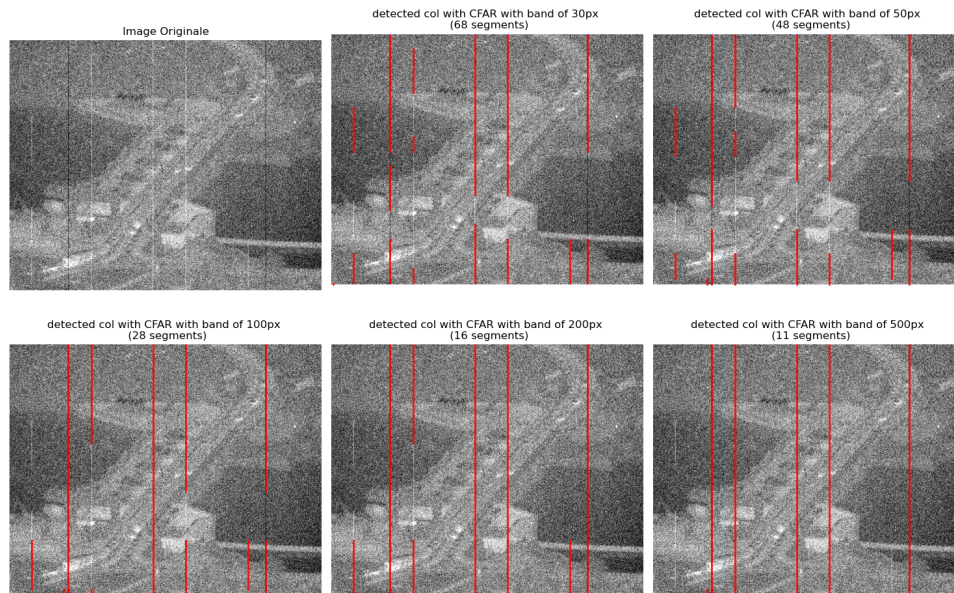


Figure 6 – Band width variation in CFAR

3.3.3 Region Growing

While methods like Local Thresholding or CFAR successfully identify the x-coordinates of defective columns, they fail to measure the precise vertical boundaries (y_{start} and y_{stop}) required for the final output. To transition from mere detection to precise measurement, a 'Region Growing' approach was implemented. This method initializes a 'seed' at a known defective pixel within a flagged column and extends the defect vertically until the healthy structural pixels of the image are reached

Two primary strategies were evaluated to define these boundaries :

1. Horizontal Comparison: Evaluating each pixel within the suspect column against its immediate left and right neighbors. This approach proved unreliable, as natural horizontal gradients and standard image noise frequently triggered false stops.
2. Vertical Gradient Tracking: Analyzing the intensity transitions vertically along the column itself. The underlying physical principle is that within a fragmented defect, adjacent vertical pixels share similar abnormal intensities. The transition from the defect back to a healthy pixel creates a sudden, sharp jump in intensity.

A critical challenge was defining the threshold that triggers the algorithm to stop growing. Initially, computing a stopping threshold based on the median of raw pixel intensities was considered. However, this risked failing on small fragmented defects, as their local median intensity might closely resemble the healthy background.

The solution was to shift from absolute intensities to gradients. The threshold is now calculated based on the median of the differences between consecutive pixels. Therefore, the stopping criterion is not

triggered by a specific intensity value, but by an abnormal jump relative to the typical local vertical gradient.

In practice, the algorithm operates as follows :

1. A starting point (the "seed") is identified within the suspect column, representing a confirmed defective pixel.
2. A local baseline is established by calculating the typical vertical intensity variation (the natural gradient) along the column.
3. A dynamic stopping threshold is defined, representing an abnormal intensity jump that is significantly higher than the column's natural gradient.
4. Starting from the seed pixel, consecutive pixels are evaluated iteratively to expand the segment both upwards and downwards.
5. The expansion halts automatically in a given direction as soon as the absolute intensity difference between two consecutive pixels strictly exceeds the defined threshold.
6. The final upper and lower coordinates reached by the expansion are recorded as the definitive vertical boundaries (y_{start} and y_{stop}) of the defect.

Strengths and Limitations

Strengths : Region Growing successfully bridges the gap between 1D column detection and 2D pixel-perfect measurement. By isolating the exact vertical bounds of fragmented defects, it prevents the correction algorithms from modifying healthy pixels, thus drastically improving the $RMSE_{ok}$ score (see Figure ??).

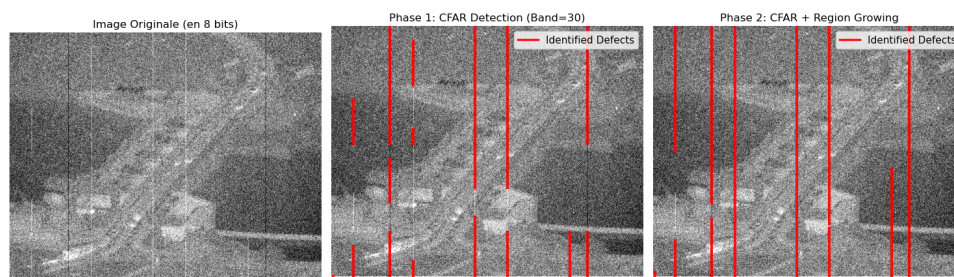


Figure 7 – CFAR with Region Growing

Limitations : The accuracy of the Region Growing heavily depends on the seed placement and the tolerance multiplier. If the tolerance is set too high, the algorithm acts is over-extending past the defect into healthy areas (see Figure 8. If set too low, it stops prematurely at the slightest internal noise variation within the defect itself. This high sensitivity necessitated rigorous hyperparameter tuning for each specific sensor type.

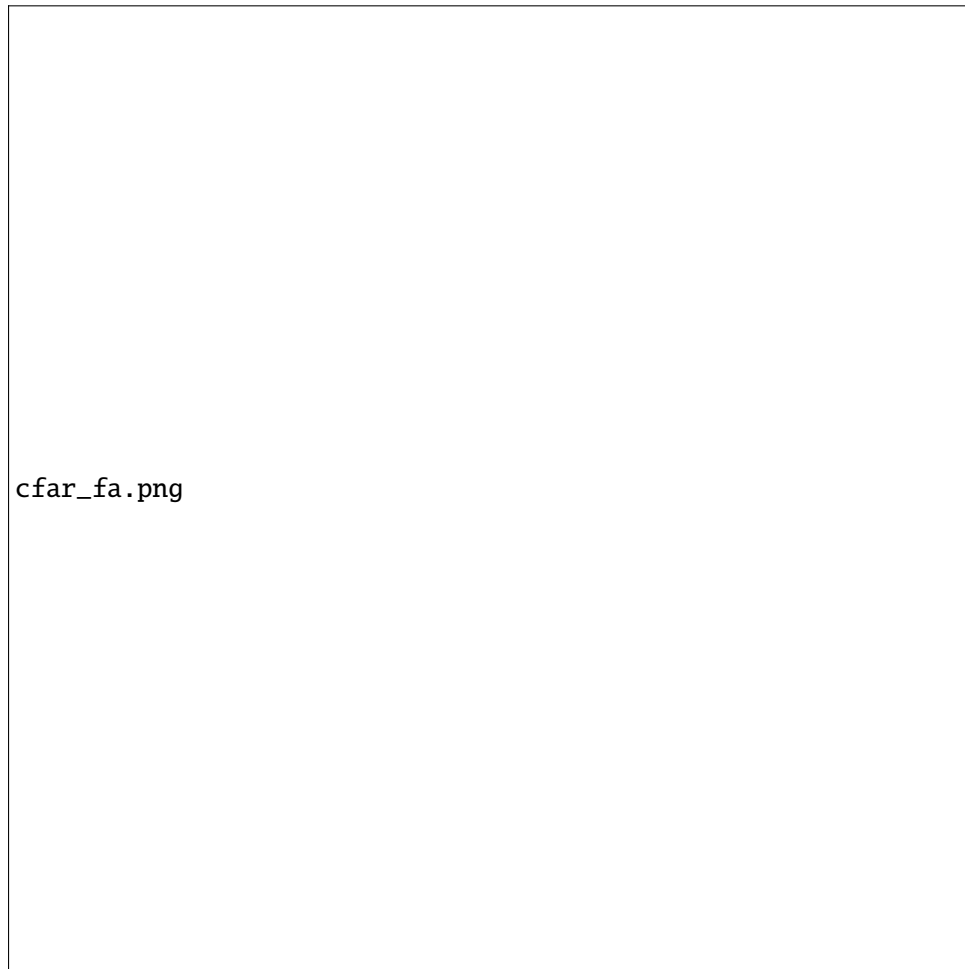


Figure 8 – CFAR with Region Growing

3.3.4 Machine Learning : Random Forest / XGBoost

Rather than evaluating our custom techniques in isolation and discarding all but one, we chose to combine their underlying principles using a Machine Learning approach. To achieve this, we transitioned through two distinct phases : establishing a baseline with a standard algorithm and broad statistical features, and then optimizing the model using advanced boosting techniques and highly targeted custom features. We create a baseline for every camera-type.

Random Forest :

Initially, we trained a preliminary baseline model to assess the general feasibility of classifying defective columns. We selected the Random Forest algorithm, a choice guided by strong consensus within the data science community.

Random Forest is an ensemble learning method built upon decision trees. A single decision tree classifies data by recursively splitting it at various nodes based on specific feature thresholds (e.g., checking if a column's mean intensity is strictly under 450). However, relying on a single tree often leads to overfitting, where the model memorizes the training data but fails to generalize. To mitigate this risk, our model utilized an ensemble of 150 independent trees. Each tree was trained on a random subset of data and features, generating its own rules. The final classification was then determined by a majority vote across all trees.

To feed this initial model, we extracted a broad spectrum of standard statistical features to see how the

algorithm would perform without our custom detection methods. We use some features such as the mean, the standard deviation, min and max, the range, etc...

XGBoost Algorithm :

While the Random Forest baseline proved the concept, we needed to push the predictive performance further. We transitioned to XGBoost (eXtreme Gradient Boosting), an algorithm recognized and famous in Kaggle Competition.

Unlike Random Forest, which builds independent trees simultaneously, XGBoost builds trees sequentially. Each new tree is explicitly trained to correct the residual errors made by the previous ones. This gradient boosting approach drastically improved our detection accuracy.

Simultaneously, we abandoned the " approach of using generic statistical features. Instead, we injected the core logic of our custom spatial and temporal algorithms directly into the model. By vectorizing image processing, we extracted five highly optimized features that capture the exact spatial and temporal anomalies characteristic of defective columns :

- Local Trend Deviation (`ecart_lt_moy`): The absolute difference between the column's mean and a localized rolling average (computed via a highly optimized sliding window). This captures spatial isolation.
- Normalized Local Ratio (`ratio_lt_moy`): The deviation from the local trend divided by the local standard deviation, acting as a dynamic Z-score that adapts to the local noise floor.
- Spatial Gradient (`spatial_diff`): The absolute difference between the column's mean and the average of its immediate left and right neighbors.
- Absolute Temporal Difference (`diff_temp_absolue`): The absolute difference in mean intensity between the current frame and the previous frame, detecting sudden temporal shifts.
- Temporal Flicker (`scintillement_temporel`): The absolute difference between the column's current mean and the average of the previous and next frames, effectively isolating high-frequency temporal noise (blinking columns).

Strenghts and Limitations :

Strenghts : By combining the sequential error-correction of XGBoost with these five targeted spatial-temporal features, the model successfully synthesized the strengths of our previous signal-processing methods into a single, highly accurate predictive pipeline. Giving us a very good Recall.

Limitations : First, the risk of overfitting remains a concern. Because we train our models on specific sequences and evaluate them using metrics derived from those same types of sequences, the model might memorize specific noise patterns rather than generalizing perfectly. Second, due to time and computational constraints, we did not perform an exhaustive hyperparameter tuning of the ML architectures themselves; parameters such as the maximum tree depth (layers) or the exact number of estimators were set to standard baselines rather than fully optimized. Furthermore, while our five custom features proved highly effective, the feature space is not fully exhausted. Exploring additional advanced spatial or frequency-domain features could potentially refine the predictions even further. Finally, this method tends to generate more False Positives (FP) compared to traditional statistical approaches.

3.4 Hyperparameter Optimization

Because the VGA, HD, and SXGA datasets feature drastically different resolutions and noise profiles, a universal parameter configuration resulted in poor and inconsistent performance depending on the evaluated sequence. Key variables, like sliding window sizes, standard deviation multipliers, and Region Growing tolerances, required sensor-specific tuning.

Optimization Strategy

Traditional tuning methods were discarded. A Grid Search approach proved computationally unviable (e.g., evaluating 4 parameters with 5 potential values requires $5^4 = 625$ tests by image), and Random Search lacked directional efficiency. Instead, Bayesian Optimization was implemented using the Optuna Python library. Far from blind searching, Bayesian optimization learns from prior iterations. By mapping the performance of previous trials, the algorithm statistically directs future tests toward the most promising value ranges, converging rapidly on the optimal hyperparameter configuration.

In practice, the algorithm :

1. An objective function is defined to evaluate the detection pipeline, returning a composite score that balances the F1-score, $RMSE_{def}$, and $RMSE_{ok}$.
2. A search space (minimum and maximum boundaries) is established for each target hyperparameter.
3. The algorithm iteratively executes the pipeline. After each trial, it updates a probabilistic model to predict and select the parameter combination that maximizes the expected improvement for the next iteration, until the absolute maximum score is reached.

3.5 Final Detection Architecture in 2 steps

The final detection architecture was structured into two distinct sequential phases: identify first, measure second.

Phase 1 :

Instead of evaluating the entire image pixel-by-pixel, this initial phase isolates only the x -coordinates of the anomalous columns. Depending on training time and computational constraints, two parallel methods are deployed :

- Local Thresholding (LT): Serves as a fast, statistical baseline to flag columns based on local neighborhood deviations (as detailed in Section 3.3.1).
- Random Forest Classifier / XGBoost: A machine learning alternative trained to recognize the exact statistical signatures of the specific defect profiles, providing highly robust column identification (as detailed in Section 3.3.4).

Phase 2 :

Once a column is flagged, the Region Growing algorithm (detailed in Section 3.3.3) is applied to extract the exact vertical boundaries (y_{start} , y_{stop}). By planting a seed at the epicenter of the defect and expanding vertically until a natural structural gradient is reached, this phase isolates the defect.

This two-step approach ensures that the subsequent correction stage targets only the damaged pixels while fully preserving the $RMSE_{ok}$ score.

However, it is worth noting that relying solely on our Machine Learning model would have been a highly viable, and perhaps more practical, alternative. While XGBoost produces slightly more False Positives (meaning it occasionally flags healthy columns as defective), visual inspections revealed that applying our correction algorithms to a few healthy columns is generally imperceptible to the naked eye. In a real-world industrial scenario, prioritizing XGBoost's near-perfect recall—ensuring absolutely no defect is missed—over strict mathematical perfection might actually be the preferred choice.

4 Correction

After detecting the defective columns and fragmented stripe regions, the next step of the project was to correct only the damaged pixels while preserving the original image information as much as possible. Since the defects were not always full column stripes and could also appear as fragmented vertical segments, the correction stage had to be local, selective, and compatible with the defect coordinates stored in the JSON file.

The correction process was designed around one main principle: only the detected defective pixels should be modified, while all clean pixels should remain unchanged. This is important because infrared images can contain subtle local details, and applying a correction to the whole image could unnecessarily alter regions that are already correct.

4.1 Smoothing

To correct the defective pixels identified during the detection phase, our initial approach focused on 2D local interpolation using surrounding pixels. To systematically test this concept, we developed an algorithm. This modular function was designed to combine various spatial shapes with different weight distributions and mathematical aggregation methods.

We experimented with four main shapes: Cross, Square, Circle, and Bowtie, each tested at different scales (3x3, 5x5, and 7x7 pixels). These shapes were then coupled with weight matrices: Uniform (all neighbors hold equal weight), Gaussian (weights decrease smoothly from the center), and Pyramid (linear decrease). Finally, the selected neighboring pixels were aggregated using different mathematical methods: mean, weighted average, median, or trimmed mean (to exclude extreme values).

While this extensive testing matrix yielded mathematically good numerical scores on some sequences, the visual results were unsatisfactory. The corrections often left defects. Moreover, the performance was completely unstable across different images; ironically, shapes specifically designed to demonstrate poor performance : the simple cross, sometimes returned the best scores on certain frames.

From this ironic observation, we attempted to model and quantify the electronic defect generated by the faulty amplifiers, hoping to mathematically subtract it from the active pixels. We quickly abandoned this path, as it proved impossible to accurately measure and isolate the defect's coefficient signature. Furthermore, we could not confirm whether the error amplification coefficients were strictly identical across a single defective column.

Ultimately, the lack of visual quality and stability of the 2D local methods definitely oriented our strategy towards strictly lateral (1D) correction techniques, such as horizontal frequency interpolation, which proved to be far more robust and visually consistent.

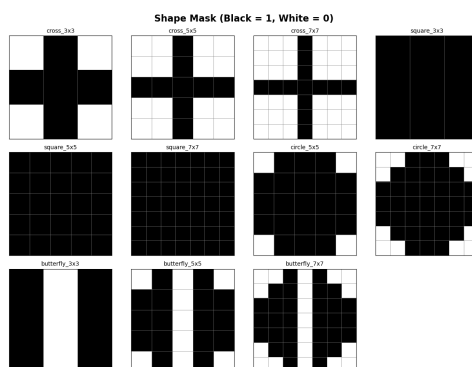


Figure 9 – Table of the different shapes used for a smoothing filter, where white pixels equal 0 and black pixels equal 1

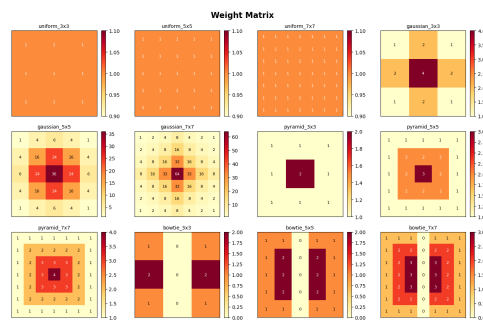


Figure 10 – Table of the different weights used for a smoothing filter



Figure 11 – From left to right: Image with defect, Image corrected via the smoothing method, Reference image without defect

4.2 Copy paste method

This algorithm applies a highly straightforward approach: it replaces defective pixels with the values of their immediate right neighbors, processing columns from right to left to propagate the fix. Thanks to its extreme algorithmic simplicity, this is one of our most lightweight and computationally efficient methods. Visually, the corrections look surprisingly excellent on isolated defects, even though the numerical evaluation scores remain quite poor. However, its major limitation becomes apparent when dealing with multiple consecutive defective columns: the repeated duplication of the same pixel creates a noticeable "smearing" or stretching artifact on the final image.



Figure 12 – From left to right: Image with defect, Image corrected via the smoothing method, Reference image without defect

4.3 Inpaint Opencv

To explore more advanced image restoration techniques, we implemented a correction method leveraging OpenCV's built-in Navier-Stokes inpainting algorithm (`cv2.INPAINT_NS`). This approach works by first generating a strict binary mask that isolates all the defective pixels. The algorithm then treats the image intensity as a fluid dynamics problem. Using partial differential equations, it calculates the gradients at

the boundaries of the mask and smoothly propagates lines of equal intensity from the surrounding healthy pixels inward to bridge the gap.

Despite the mathematical sophistication of this technique, the overall numerical evaluation scores were disappointing. However, visual inspections revealed a striking discrepancy based on the defect's nature: the algorithm performed significantly better on dark defective pixels, successfully blending them into the background. In sharp contrast, it struggled with bright or light defects, where the correction failed to look natural, leaving jarring smudges and ghosting artifacts that were clearly visible to the naked eye.



Figure 13 – From left to right: Image with defect, Image corrected via the Opencv's built-in Navier-Stokes inpainting method, Reference image without defect

4.4 Non-Uniform Correction method for dealing with Fixed Pattern Noise

To address the limitations of standard morphological operations, a Non-Uniformity Correction (NUC) method designed to tackle Fixed Pattern Noise at its physical source was implemented. Recognizing that these column defects stem from amplifier malfunctions, it was hypothesized that the errors must share a common hardware signature. The core idea was to construct a global "master template" capturing this unique error profile by comparing defective images from the training set against their ground truth. Because the dataset contains images at multiple resolutions (VGA, HD, and SXGA), all the error profiles first stretched onto a common 2048-point coordinate system to average them and extract a reliable master shape. When applying this correction to a new test image, the algorithm shrinks the template back to the target resolution. It then uses healthy neighboring columns to estimate the true scene intensity, calculates the specific deviation of the defective column, and scales the master template's magnitude to match this localized error. Finally, it smoothly subtracts this scaled signature from the column to bridge the gap. Despite the strong physical rationale behind this technique, the overall numerical evaluation scores remained below our highest expectations. While the defects were visually corrected, the algorithm's reliance on a mean scaling factor proved to be a vulnerability. It struggled significantly with noisy, blinking defects, which do not behave as simple, scaled-up versions of a fixed template. Furthermore, the method assumes that the test data amplifiers share the exact same malfunction fingerprint as the training data. To prevent this technique from inadvertently altering healthy regions when the template mismatched, a strict 2D boolean defect mask was ultimately integrated into our pipeline. This ensured that regardless of the correction generated, only the strictly defective pixels were replaced, leaving the original healthy image entirely untouched.

4.5 Interpolation

This method is inspired by the Algotom interpolation approach, which corrects gray scale values inside vertical stripes using a binary stripe mask. The general idea is to treat stripe pixels as missing or unreliable values and estimate them from surrounding healthy pixels. In our implementation, the interpolation function first creates a copy of the original image. Then, using the 2D defect mask generated from the JSON defect dictionary, it identifies the defective pixels as missing points and the clean pixels as known

points. This adapts the Algotom idea to our case, where the detected defects are not always complete vertical stripes but can also appear as fragmented vertical segments.

To improve processing speed, the interpolation is not applied to the entire image. Instead, the algorithm crops a local region around the defective columns, using a margin of five columns on each side. This reduces the amount of data to process while still keeping enough surrounding context for estimation.

The known healthy pixels inside this local region are used as reference points, and the defective pixels are corrected using 2D interpolation. Two variants are tested: linear interpolation and cubic interpolation. Linear interpolation estimates values by creating a smooth transition between neighboring known pixels, while cubic interpolation uses a higher order interpolation method that can produce smoother results in some cases. These variants are implemented using the SciPy griddata function, which supports scattered data interpolation.



Figure 14 – Result of the interpolation method on HD/sequence_1/low dyn with columns 3/frame_0043.png

The interpolation methods were simple and useful, especially for small fragmented defects. However, their performance depended strongly on the surrounding clean pixels. If the defect was large or located in a region with complex intensity variation, interpolation produced less accurate reconstruction. In the final testing, cubic interpolation obtained a score of 67%, while linear interpolation obtained a score of 70%.

4.6 Wavelet Fourier

This method works in both the wavelet domain and the frequency domain. The main idea is to reduce frequency components associated with vertical stripe artifacts without directly modifying the image only in the spatial domain. This follows the principle of combined wavelet and Fourier filtering for stripe artifact removal.

First, the image is decomposed using a 2D Discrete Wavelet Transform. This separates the image into approximation coefficients and detail coefficients, including horizontal, vertical, and diagonal components. Since the defects in this project are vertical stripe artifacts, the method focuses mainly on the vertical detail coefficients. The wavelet decomposition and reconstruction steps are implemented using PyWavelets, a Python package for wavelet analysis.

For each decomposition level, a Fourier transform is applied to the vertical detail coefficients. A Gaussian damping filter is then used to reduce stripe related frequencies. After filtering, the inverse Fourier transform and inverse wavelet transform are applied to reconstruct the corrected image. As with the previous methods, only the values inside the defect mask are copied into the final output, while clean areas are preserved.



Figure 15 – Result of the interpolation method on VGA/sequence_1/low dyn with columns 2/frame_0200.png

In the experiments, the Wavelet Fourier method showed limited performance. It obtained a score of 86% for VGA, 80% for SXGA, and 68% for HD. Visually, it slightly improved the VGA images, but it could not correct many defective columns in the SXGA and HD images. Therefore, although this method is theoretically suitable for stripe suppression, it was not reliable enough for our dataset.

4.7 Frequency Interpolation

This is a hybrid frequency interpolation method inspired by Algotom style destriping ideas. This method is designed to combine frequency separation with local interpolation. The general correction idea is inspired by Algotom, where detected stripe regions can be corrected through interpolation. It is also related to Sarepy style correction, where stripe artifacts are handled by separating and correcting smooth intensity components.

The image is first converted to float32 format. Then, a vertical Gaussian filter is applied along the vertical axis. This produces a low frequency image component. The low frequency component contains the smooth background information and the slowly varying part of the stripe defects. The high frequency component is obtained by subtracting the low frequency image from the original image. This separation is important because with the vertical Gaussian, stripe artifacts are more likely to appear in the low frequency component, while fine details such as edges and local texture are mainly preserved in the high frequency component. Therefore, the method corrects only the defective parts of the low frequency component instead of directly modifying the whole image.

For each image row, the algorithm identifies defective columns using the exact defect mask. The clean columns on the same row are used as reference points, and the defective columns are filled using 1D horizontal linear interpolation. This step is implemented using the `np.interp` function, which performs one dimensional linear interpolation from known sample points [6]. This means that the algorithm estimates the damaged low frequency values from the clean pixels on the left and right side of the stripe.

After correcting the low frequency component, the original high frequency component is added back. This fusion step restores the fine image details while correcting the smooth part affected by stripe defects. Finally, the corrected image is clipped to the 16 bit intensity range, and only the masked defective pixels are copied into the final output. All clean pixels remain exactly the same as in the original image.

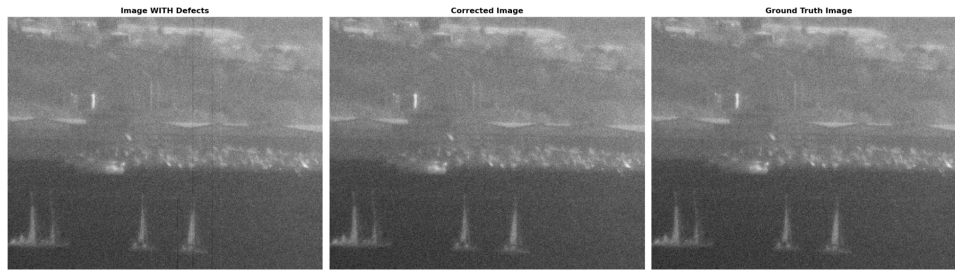


Figure 16 – Result of the interpolation method on VGA/sequence_3/low dyn with columns 3/frame_0258.png

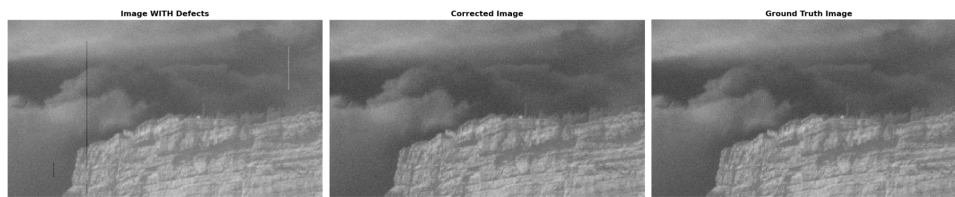


Figure 17 – Result of the interpolation method on HD/sequence_3/low dyn with columns 2/frame_0258.png



Figure 18 – Result of the interpolation method on SXGA/sequence_3/low dyn with columns 3/frame_0138.png

This method achieved the best result among the tested classical correction methods, with a score of 94%. Its strong performance can be explained by the fact that it corrected the low frequency stripe component while preserving high frequency image details. Therefore, it provided a good balance between stripe removal and detail preservation.

4.8 Trials of Deep Learning Models

In addition to classical image processing methods, two deep learning models were also tested as possible directions for stripe correction: ASCNet and DestripeCycleGAN. These models were explored because deep learning methods can potentially learn complex stripe patterns and reconstruct defective regions more intelligently than traditional image processing methods.

The first tested model was ASCNet, or Asymmetric Sampling Correction Network. This model was proposed for infrared image destriping and was designed to capture global column relationships in infrared images [7]. After testing it on our images, the model was able to remove part of the stripe artifacts. However, the improvement was limited. The difference between the original defective image and the corrected image showed that the model removed approximately 28% of the stripe information.

This suggested that ASCNet had some correction ability, but it was not strong enough to be selected as the final method.

The second tested model was DestripeCycleGAN, an unsupervised infrared image destriping method based on CycleGAN and stripe simulation [8]. It produced some visible stripe reduction. Since our images contained both bright and dark stripe defects, several processing directions were tested. The image was first processed directly to target bright stripes. Then, an inverted version of the image was tested to target dark stripes before being inverted back after correction. A combined approach was also explored. Among these trials, the inverted image strategy appeared visually better, but the model still had difficulty correcting fragmented stripe defects and did not produce stable results.

Overall, the deep learning trials were promising but not suitable as the final correction method for this project. The main limitation was that the available pretrained weights were not trained on our own infrared images. Therefore, the models had difficulty generalizing to our specific defect patterns, especially fragmented stripes and mixed black and white stripe artifacts. In addition, the tested models were mainly designed for complete column stripe correction, while our data contained both complete columns and local defective segments.

Another limitation was the available project time. Training or fine tuning a deep learning model properly would require a larger dataset, ground truth corrected images, parameter tuning, and longer training time. Since the correction stage had to be completed within a limited period, deep learning was kept as a tested direction rather than the selected final solution.

5 Results

6 Limitations Future Perspective

During the development of our machine learning strategies, dealing with the risk of overfitting remained a primary constraint. Because our dataset consists of video sequences, the high temporal correlation between consecutive frames introduces a significant risk of data leakage. To ensure an unbiased evaluation and prevent our models from merely memorizing the scenes, we enforced a strict validation strategy: models were trained exclusively on two sequences, while the third was entirely isolated for final testing. A notable limitation of this study lies in the simulated nature of the anomalies. While we initially explored the possibility of generating our own augmented datasets, accurately reproducing the intricate, hardware-driven noise proved excessively complex. Gaining access to a broader corpus featuring natural defects would be the next step to push the robustness of our algorithms even further.

Finally, our exploratory tests with machine learning architectures, such as XGBoost, highlighted a striking discrepancy between strict mathematical metrics and human visual perception. While these predictive models demonstrated exceptional recall by detecting nearly all defects, they also introduced a marginal number of false positives. The current scoring metric penalizes these misclassifications, driving down the overall numerical performance. However, visual inspections revealed an interesting paradox: accepting the trade-off of inadvertently smoothing a few healthy pixels actually produces an output that is significantly cleaner and more natural to the naked eye. This observation suggests that strictly minimizing numerical error may occasionally come at the expense of overall perceived image quality.

7 Appendix : Detection Performance Metrics

Table 1 – **Baseline Metrics Comparison.** Performance comparison of baseline defect detection methods (Local Thresholding (LT), LT by band, CFAR by band), and statistical metrics (Mean, Median, Standard Deviation/Variance) on a single VGA sequence (Sequence 1, Column Type 3). The results highlight the superiority of central tendency metrics (Mean) over variance-based metrics for baseline Local Thresholding.

Method	Metric	Precision	Recall	F1-Score
LT	Mean	100.00 %	90.87 %	95.21 %
LT	Median	100.00 %	87.02 %	93.06 %
LT	Std Dev	2.76 %	50.99 %	5.24 %
LT by band	Mean	36.26 %	96.09 %	52.65 %
LT by band	Median	18.56 %	96.16 %	31.12 %
LT by band	Variance	1.02 %	80.72 %	2.02 %
CFAR by band	N/A	92.48 %	88.64 %	90.52 %

Table 2 – **Impact of Optimization on Specific Sequences.** Impact of hyperparameter optimization (via Optuna) on the F1-Score for targeted sensor sequences. The optimized parameters demonstrate significant performance gains, particularly for initially underperforming baseline configurations.

Type & Sequence	Method	Initial F1-Score	Optimized F1-Score	Optimized Parameters
VGA - Seq 1	LT	95.21 %	95.84 %	Window: 150, Std: 1.00
VGA - Seq 1	LT by band	52.65 %	96.57 %	Window: 25, Std: 2.12, Bands: 4
VGA - Seq 1	CFAR by band	90.52 %	94.26 %	Band: 116, Train: 8, Guard: 3, Mult: 3
VGA - Seq 2	LT	30.25 %	85.61 %	Window: 16, Std: 2.07
VGA - Seq 2	LT by band	3.46 %	85.45 %	Window: 16, Std: 3.42, Bands: 4
VGA - Seq 2	CFAR by band	43.30 %	81.79 %	Band: 134, Train: 9, Guard: 2, Mult: 4
SXGA - Seq 2	LT by band	14.25 %	70.67 %	Window: 15, Std: 1.91, Bands: 5
HD - Seq 2	LT by band	25.24 %	59.25 %	Window: 54, Std: 2.07, Bands: 4

Table 3 – **Optimization by Image Type.** Global performance evaluation of three defect detection methods (Local Thresholding, Strip-based LT, and LT with Region Growing) across all sequences for VGA, SXGA, and HD sensors. The table details the optimized hyperparameters and highlights the resulting F1-Scores for each configuration using the Mean as the baseline metric.

Type	Method	Optimized F1-Score	Optimized Parameters
VGA	LT	72.53 %	Window: 18, Std Factor: 1.92
VGA	LT by band	72.04 %	Window: 20, Std Factor: 1.80, Bands: 1
VGA	LT with RG	80.46 %	break mult: 6, smooth size: 21
SXGA	LT	62.40 %	Window: 31, Std Factor: 1.74
SXGA	LT by band	70.53 %	Window: 29, Std Factor: 2.81, Bands: 7
SXGA	LT with RG	74.37 %	break mult: 6, smooth size: 15
HD	LT	53.46 %	Window: 24, Std Factor: 1.26
HD	LT by band	46.69 %	Window: 26, Std Factor: 2.84, Bands: 8
HD	LT with RG	76.08 %	break mult: 3.8, smooth size: 3

Table 4 – **Detection Scores with XGBoost.** Detailed performance metrics using the XGBoost `classification_prediction` model across VGA, SXGA, and HD sensor resolutions. The table highlights precision, recall, and F1-Scores for both healthy (Class 0) and defective (Class 1) columns.

Sensor	Category	Precision	Recall	F1-Score	Support
VGA	Class 0 (Healthy)	0.96	0.99	0.97	49,244
	Class 1 (Defective)	0.98	0.92	0.95	24,622
	Accuracy	–	–	0.97	73,866
	Macro Avg	0.97	0.95	0.96	73,866
	Weighted Avg	0.97	0.97	0.97	73,866
SXGA	Class 0 (Healthy)	0.94	0.93	0.94	148,968
	Class 1 (Defective)	0.87	0.89	0.88	74,484
	Accuracy	–	–	0.92	223,452
	Macro Avg	0.90	0.91	0.91	223,452
	Weighted Avg	0.92	0.92	0.92	223,452
HD	Class 0 (Healthy)	0.94	0.95	0.94	83,760
	Class 1 (Defective)	0.90	0.88	0.89	41,880
	Accuracy	–	–	0.93	125,640
	Macro Avg	0.92	0.91	0.92	125,640
	Weighted Avg	0.93	0.93	0.93	125,640